

The Emulator II Computer Music Environment

Dana C. Massie
E-Mu Systems
Santa Cruz, CA

Abstract

A real time sampling musical instrument is described which is combined with a non real-time software analysis, synthesis, and resynthesis system. The sampling instrument functions include user sampling of waveforms, real-time keyboard playback and transposition of sounds, looping of waveform segments, a velocity sensitive keyboard, and real-time filtering and enveloping of polyphonic output. Non real-time software functions include waveform display and editing, phase vocoder spectrum analysis and resynthesis, and synthesis using several well-known computer music algorithms.

1. Emulator II System Hardware

The Emulator II is a polyphonic digital sampling instrument. The important system elements are shown in Fig. 1. Digitized sound data are stored in the 512 KByte dynamic random access memory. Access to memory is shared by a Z-80 microprocessor and eight D-A converters, with arbitration performed by a high-speed microprogrammed bipolar microcontroller. The microcontroller acts as an eight channel DMA controller, with facilities for looping, and incrementing or decrementing sound-memory addresses for playing a sound either forwards or backwards.

Each of the eight analog output channels contains a D-A converter, a tracking 7th-order elliptical low-pass reconstruction filter, a 4-pole voltage-controlled lowpass filter (VCF), and a voltage-controlled amplifier (VCA). The VCA's allow changing the envelope of the sound in real-time, and the VCF's allow altering the spectrum in real-time. Separate 4-phase ADSR envelopes are supplied for each VCA and VCF.

All Emulator II's have either one or two 5-1/4 " floppy disk drives for storage of audio data, system software, and musical note sequences. A fast Winchester hard disk is available as an option. The Emulator II also incorporates an Analog-to-Digital converter for user sampling (digitizing) of sounds. For external interfacing, both a standard MIDI port and a high-speed (500-KBaud) RS-422 computer port are included. The front panel of the Emulator II has a set of four slider pots

interfaced to the microprocessor, and set of push-button switches for control of system functions.

2. Emulator System Software

The basic data structure in the Emulator II is the "voice", which is an arbitrary length of audio data and an associated 256 byte structure called a **voice descriptor**. The voice descriptor contains the pointers to the sound data start and stop addresses, truncate stop and start address (how much of the sound is actually played), loopstart and size parameters, and a set of VCF and VCA parameters. A single set of sound data can have multiple voice descriptors associated with it, to provide multiple timbres from a given starting point. For example, a piano sample may have one voice with a normal attack, and another with a slow attack generated by the VCA and envelope generators.

Within a voice, the keyboard velocity can be mapped on to seven separate parameters; VCA Amplitude, VCA Attack Time, VCF Amplitude, VCF Attack Time, VCF Q (Resonance), and Velocity Crossfade and Velocity Switch. These last two are significant, because they allow either a crossfade between the primary voice of a key and its secondary voice, or a selection between them as a function of velocity. This combination of mappings provides significant expressive power, in fact, more than many synthesis instruments which allow real-time input to a synthesis algorithm.

Once created, a voice is then assigned to the keyboard. Each voice can be transposed in pitch by up to two octaves. Each key on the keyboard can have two voices assigned to it, a **primary** and a **secondary** voice. Up to 99 voices are available to the user, so each key can have a completely different set of voice parameters, providing full poly-timbral keyboard splitting capabilities. In practice, seldom more than 15 or 20 different voices are used to cover a keyboard. As as few as three voices can cover the keyboard. For convincing instrument simulations, a large number of voices (15) might be necessary if the instrument has a strong formant structure which sounds unnatural when pitch-shifted. If the instrument does not have a strong formant, only a few voices are needed.

The mapping of voices to keyboard is a data structure called a **Preset**. Switching between presets is essentially instantaneous, allowing rapid access to different timbres. A preset also contains the parameters controlling MIDI, arpeggiator settings, and real-time controls such as the modulation wheels and the footswitches and pedal. The arpeggiator can be assigned to be active on only a subset of the keyboard. Since each preset has its own MIDI assignments, switching between presets allows switching between different MIDI channels, to rapidly change which MIDI device is being controlled from the Emulator.

The collection of all presets in memory is called a bank and is shown schematically in fig. 2.

Defining a voice by a user is a simple job, and the underlying detailed data structures are transparent to the user. Parameters are edited using a fast tree structured menu system with extensive prompting from the two-line LCD display. The editing is fully interactive, with immediate feedback available by playing the keyboard to hear sounds.

3. Macintosh / Digtidesign Software

The Digtidesign Software package called **Sound Designer** provides a full set of waveform display, analysis, modification, and synthesis functions. Digital audio data can be transferred between the Emulator and the Macintosh at 500 KBaud, and then displayed or processed in non-real-time. In fig. 3, about 100 msec of a Kalimba waveform are shown in a **Sound Designer** screen image. The waveform display can be scrolled smoothly and quickly through the sound file, and the scale factor can be changed to provide single sample resolution (as in fig. 4 the first 10 mSec of the same waveform), or to show the envelope of the entire sound file. Accurate time and amplitude scale calibration marks are also provided. Complete use of the Macintosh paradigms for selection and editing are provided to enable cut and paste of waveform segments selected by simple use of the mouse. Up to three windows can be open at one time, each containing a different waveform. A set of mixing, merging, and digital signal processing functions are provided.

A time-frequency analysis of the same Kalimba is shown in figure 5. This is a 256 point FFT, implemented with a radix-2 decimation in time algorithm, using a hop factor of 2-to-1. The three dimensional spectrum display incorporates hidden line removal, with most display parameters controllable by the user. This is the first stage in the development of a full weighted-overlap-add phase vocoder implementation (Portnoff 1975).

Integer arithmetic is used to to minimize processing time, but block floating point is used where necessary

to ensure maximum signal-to-noise ratio with the least amount of computational overhead (Tran-Thong, Liu 1976). As a benchmark, a 256-point complex FFT executes in under 150 msec, using block floating point.

Currently, the Karplus-Strong plucked string algorithm (Karplus, Strong 1983) is implemented to allow direct digital synthesis of sounds. Several other synthesis techniques are planned, notably non-linear waveshaping, F.M., and additive (Arfib 1979, LeBrun 1979, Chowning 1973). Since the synthesis of sounds need only be done for short sound events (typically only one or two seconds are needed for a good Emulator II sample), non-real-time synthesis is very fast and efficient. Once the sound has been created, it can be played in real-time on the Emulator II. This provides an excellent compromise between the flexibility of non-real-time synthesis and the efficiency of real-time performance.

Sound files are a standard 16-bit linear format, so transfer between Sound Designer and user written processing and synthesis programs at the file level will be available. This way, full use of Sound Designer features will be available to users who want to write their own algorithms.

Editing of Emulator II parameters can be performed through the Sound Designer package, as shown in the Sound Designer screen images in figs. 6, 7 and 8. Graphical displays assist in understanding the relationship between the parameters, and show envelope shapes and keyboard voice mappings.

4. Emulator II Philosophical Considerations

Experience with the Emulator II sheds new light on the old debate about the value of imitative synthesis. For many users, the main purpose of the Emulator II is to provide convincing simulations of traditional acoustic instruments. But for many others, producing interesting sounds is the main goal, regardless of their origin. Often a traditional instrument is used as a starting point, but modified extensively to produce a wholly new timbre, not clearly electronic or acoustic in nature. One possible benchmark for the power of a synthesis technique is the success or failure of imitative synthesis. Also, imitation is a good starting point for structuring a timbre. Many instrument families can be very convincingly imitated with an Emulator II., and in some cases, it is nearly impossible to tell the difference between an Emulator II and the recording of a performance on the original instrument. In highly modified sounds, it is also interesting how much more animated the resultant sound is compared to a typical algorithmic synthesizer, even though the Emulator II is very similar to an analog synthesizer with the oscillator replaced by digital audio playback.

4.1 Real-time vs. Non-Real-Time Tradeoffs

Traditional musical instrument design can be considered an "off-line, non-real-time" activity. Building an instrument is a "non-real-time" activity, since the instrument cannot be heard while it is being built. Once built, the instrument can be played in real-time, that is, playing produces immediately audible results.

In computer music, the division between instrument design and instrument performance is also quite appropriate. Sounds are generally designed as a separate activity, with most of the attention being given to the timbre at the microscopic acoustic level. For performance, the attention is more at the macro level.

In designing an instrument, real-time input of performance or synthesis parameters is important to give expressive control over the sound produced. In some instruments, little control is provided, such as with the harpsichord or organ. In some instruments, there are numerous powerful control parameters, such as the violin. But still, real-time control parameters can usually be thought of as distinct from non-real-time instrument design.

Constraining an electronic instrument to real-time algorithmic synthesis of audio forces severe restrictions in the flexibility and complexity of sound events. Only so much hardware can be bought with the typical musician's budget. The ability to drop out of real-time to generate sounds is a powerful extension to both flexibility and complexity. But how should the sound be played back? With only playback from disk, as in traditional software synthesis environments, there is no real-time manipulation of sounds. With the Emulator, considerable real-time manipulation of sounds are possible, especially score manipulations, one of the most common that musicians use. With the Sound Designer software, considerable flexibility in sound synthesis is available, with a minimal investment in hardware. New algorithms can be added at little additional cost, in contrast to fixed algorithm synthesizers, which cannot have new algorithms at all.

5. Conclusions

We hope that the approach of using a judicious combination of a real-time sampling instrument with non real-time processing will provide much new raw material for musical composition in an efficient and economical environment.

Acknowledgements

We would like to thank the people at the Computer Audio Research Laboratory at the Center for Music Experiment at the University of California, San Diego, especially Andy Voelkel, Mark Dolson, D. Gareth Loy, and F. Richard Moore for all their assistance. Credit should be given to Donna Murray, David Rossum, Tom Moxon, and Alan Goldwater who designed and programmed the Emulator II. Evan Brooks wrote the Sound Designer program. Finally, many thanks go to David Rossum, Scott Wedge and Marco Alpert of E-Mu Systems for their encouragement and assistance.

References

- Chowning, John M. "The Synthesis of Complex Audio Spectra by Means of Frequency Modulation," J. Audio Engineering Soc., Volume 21, Number 7, pp526-534, September 1973
- Tran-Thong, and Bede Liu, "Fixed-Point Fast Fourier Transform Error Analysis" IEEE Transactions on Acoustics, Speech, and Signal Processing, Volume ASSP-24, Number 6 December 1976
- Portnoff, Michael R., "Implementation of the Digital Phase Vocoder Using the Fast Fourier Transform," IEEE Transactions on Acoustics, Speech, and Signal Processing, Volume ASSP-24, Number 3 June 1976
- LeBrun, Marc, "Digital Waveshaping Synthesis," J. Audio Engineering Soc., Volume 27, pp250-265, April 1979
- Arfib, D., "Digital Synthesis of Complex Spectra by Means of Multiplication of Nonlinear Distorted Sine Waves," J. Audio Engineering Soc., Volume 27, pp757-768, October 1979
- Karplus, K. and A. Strong 1983 " Digital Synthesis of Plucked-String and Drum Timbres." Computer Music Journal, Volume 7 Number 2:43-55

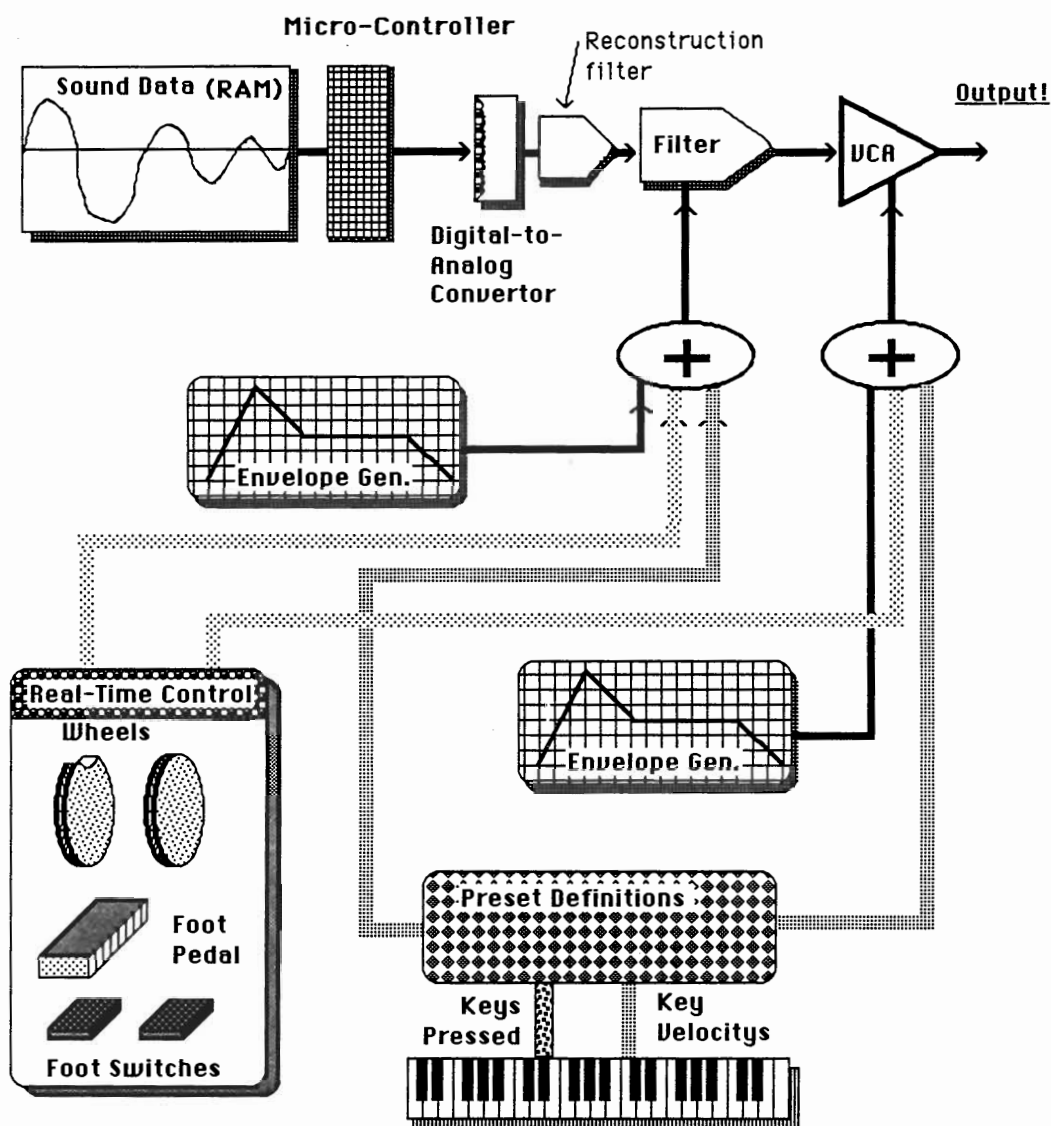


Figure 1. A single Emulator II voice shown in block diagram form. The wavetable is shared between the eight output channels.

E-II Bank Structure Block Diagram

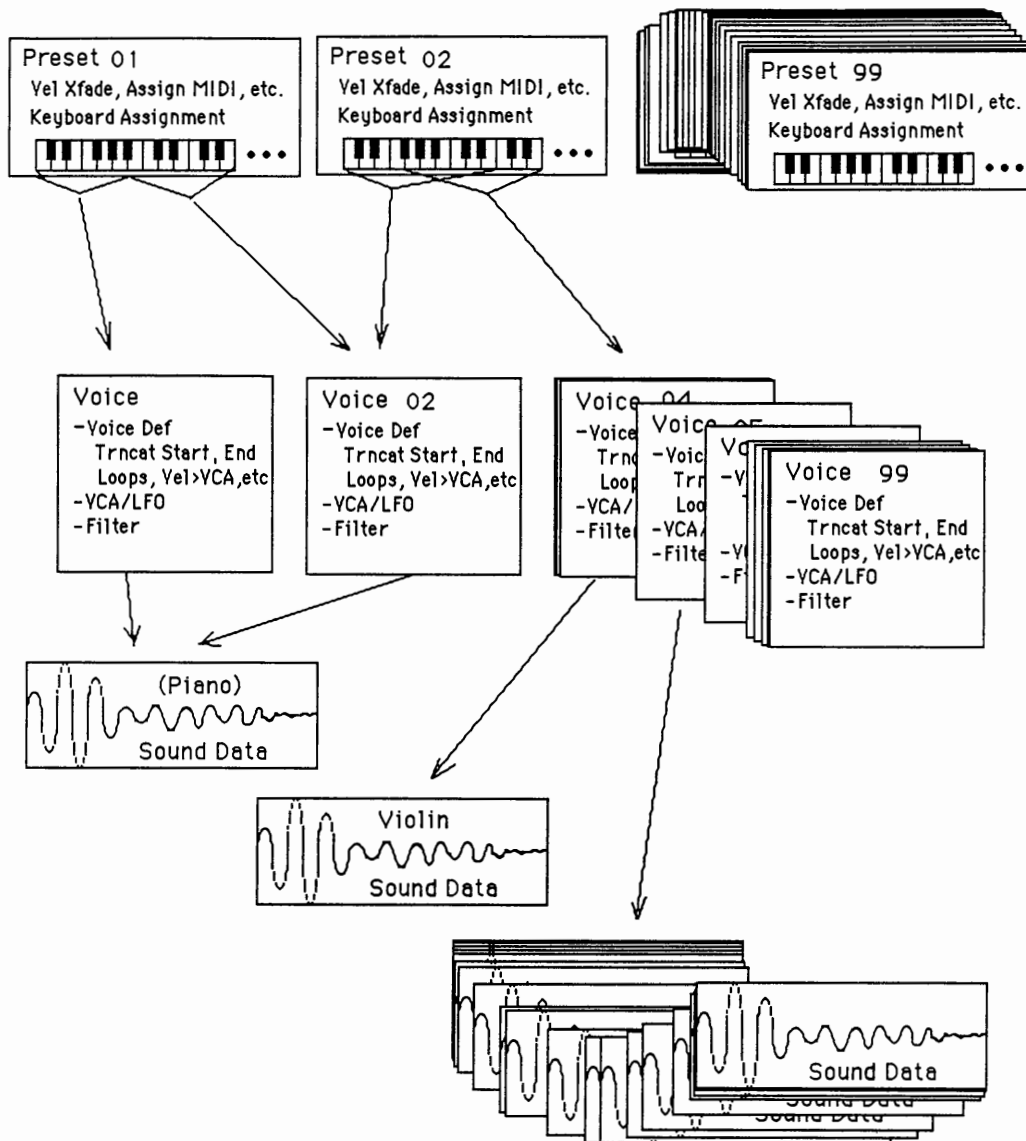


Figure 2. The relationship between Voices, Presets, and sound data is shown schematically.

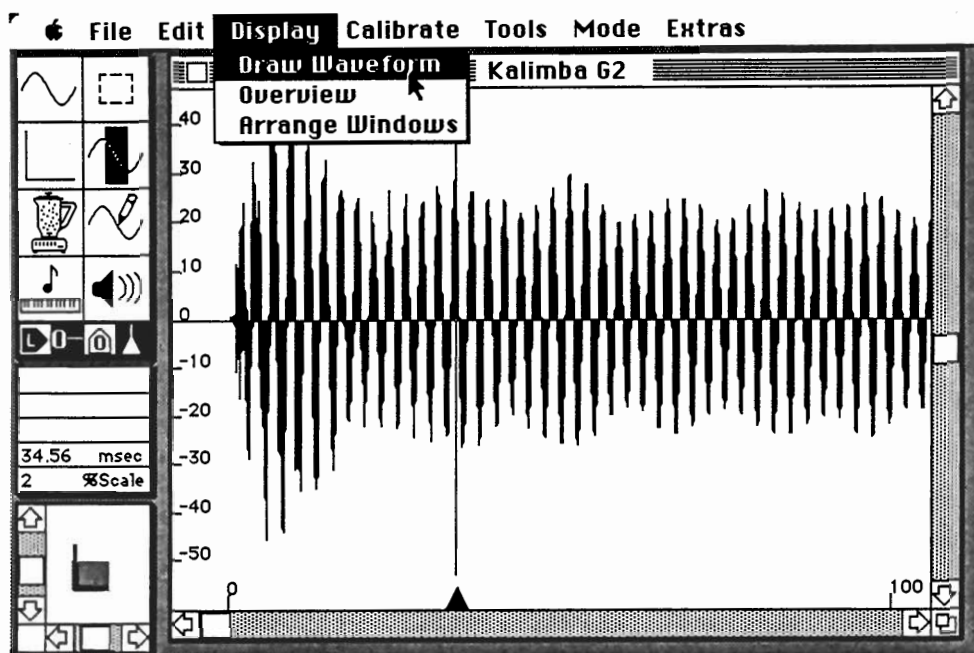


Figure 3. A waveform display of a Kalimba (African thumb piano) note. At this scale, the display is showing the envelope of the sound. About 100 msec are shown, as the calibration on the bottom of the waveform window shows.

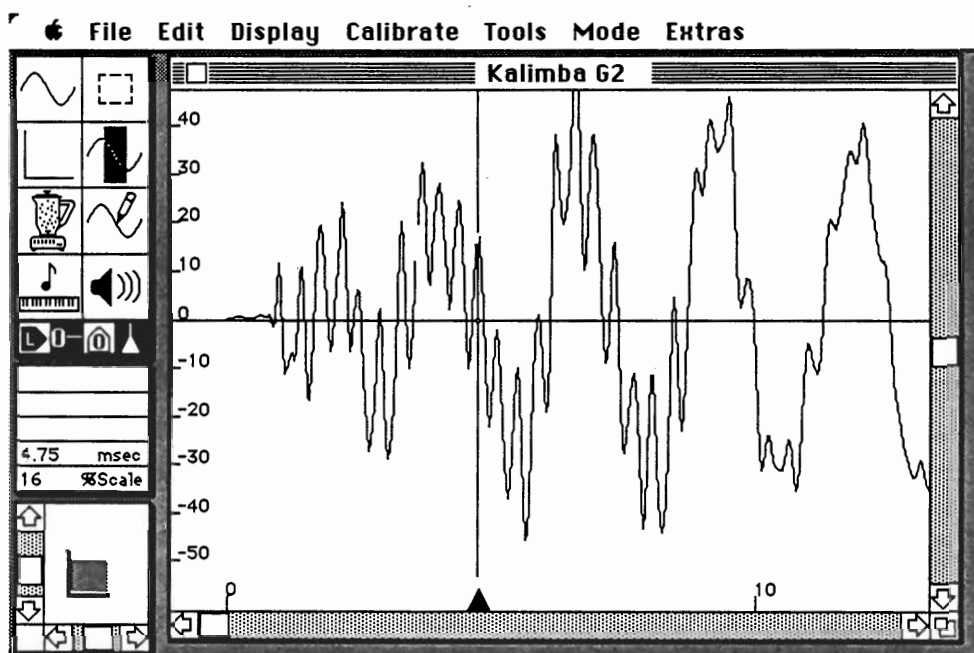


Figure 4. The first 10 msec of the same note shown in Figure 3. Here, single samples are resolved, and single sample editing can be performed with the pencil tool from the menu at upper left.

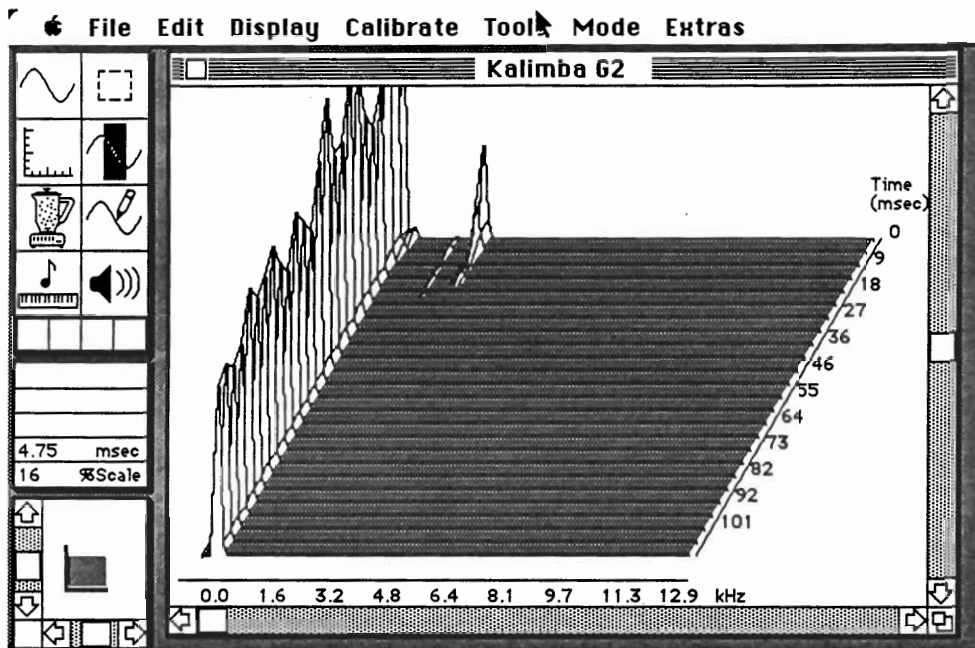


Figure 5. Spectrum analysis of the first 100 msec of the Kalimba note shown in Figs. 3 and 4.

File Edit Display Calibrate Tools Mode Extras

Preset Definition **01 Kalimba #1** ☒ Current Voice List **Look**
☐ Select Single Voice **Quit**

Memory Remaining: 49783 bytes / 484559

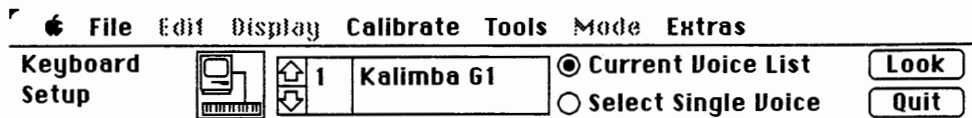
Arpeg. Mode: ☒ Off ☐ Up ☐ Down ☐ Up/Down ☐ Random ☐ Program
Extension: ☒ Normal ☐ 1 Octave ☐ 2 Octaves **Range:** C1 to C5
Note Value: ☒ 1/4 ☐ 1/8 ☐ 1/16 ☐ 1/32 ☐ 1/4 triplet ☐ 1/8 triplet ☐ 1/16 triplet ☐ 1/32 triplet
Clock: ☐ Internal ☐ External
Tempo: 200 BPM

MIDI Mode ☐ Off ☒ Omni ☐ Poly ☒ Preset Change OK ☐ Local Control **Basic Channel:** 1

Controller Assign:
Left Wheel: ☐ Off ☐ MIDI #
Rt. Wheel: ☐ Off ☒ MIDI # 1
Pedal: ☒ Off ☐ MIDI #
MIDI A: ☒ Off ☐ MIDI #
MIDI B: ☒ Off ☐ MIDI #
MIDI C: ☒ Off ☐ MIDI #

☒ Pitch ☐ Mod
☐ Pitch ☐ Mod
☐ Pitch ☐ Mod
☐ Pitch ☐ Mod
☐ Pitch ☐ Mod
☐ Pitch ☐ Mod

Figure 6. The preset definition menu from the Sound Designer package. Primarily MIDI and Arpeggiator parameters are edited in this menu.



View of Keyboard Parameters:

- | | |
|---|---|
| ☒ Primary Voices | ☐ Primary Outputs |
| ☐ Secondary Voices | ☐ Secondary Outputs |
| ☐ Positional Crossfades | ☐ Velocity Switches |
| ☐ Velocity Crossfades | ☐ Non-Transpose |

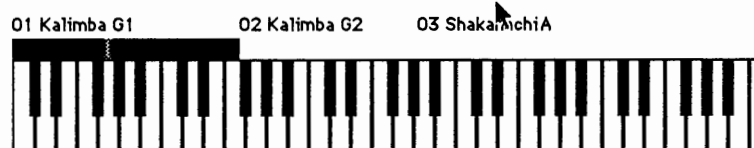
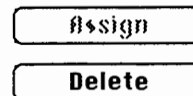


Figure 7. Keyboard Setup can be performed through this menu, with graphical displays of the positioning and range of voices assigned to the keyboard. Kalimba G1 is selected here, and its range can be seen highlighted above the keyboard icon.

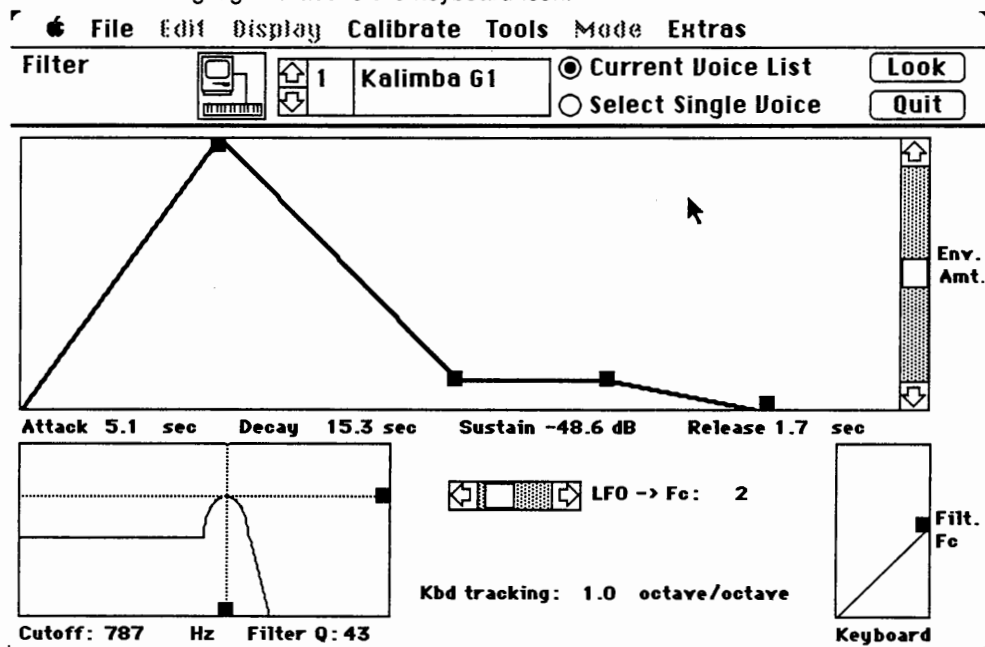


Figure 8. One of the graphical envelope generator menus; this one for the filter. The mouse is used to select a breakpoint and is dragged to change the parameter value.